

Contents

LiaScript	1
Je lui dirai démo bleueuuh...	1
LiaScript, Moodle, faut-il vraiment choisir ?	2
Un outil pensé pour l'accessibilité et le partage	3
Les 5R	3
Un outil SOUPLE	12
Un outil REMPLAÇABLE	15
Et si LiaScript venait à mourir ?	15
Mise en route	16
Se créer un compte GitHub	16
Créer un répertoire	16
Visual Studio Code	19
Git	22
Cloner votre répertoire GitHub en local	24
Finalisation de la configuration... et premier commit	28
Les bases des bases : écrire du texte	34
1. Structurer votre contenu	34
2. Mettre en forme votre contenu	35
More	36

LiaScript



Figure 1: Logo Liascript

Je lui dirai démo bleueuuh...

{{0-1}}

{{1}}

Ça s'appelle un LiaScript par récursivité, ça ne sert à rien, mais c'est POSSIBLE.

LiaScript, Moodle, faut-il vraiment choisir ?

--{{0}}--

LiaScript a pu être présenté par son créateur André Dietrich comme une alternative aux LMS¹ (cf. un article de son blog datant de 2020), ce qu'il peut être en raison de sa grande plasticité. Mais on peut tout aussi bien concevoir LiaScript comme un outil *complémentaire* des LMS tels que Moodle.

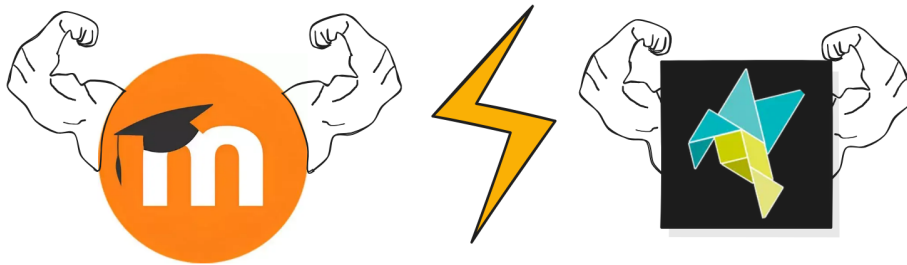


Figure 2: Who will prevail ?

Moodle, LiaScript, Scenari... et si on se faisait surtout plein de câlins ?

{{1-2}}

```
<div style="color:white; display:flex; align-items:center;">
  <div style="font-size: 1.2em;"><strong>Moodle et consort (LMS) : les bons côtés</strong>
</div>
```

```
<ul>
  <li> <span style="font-size: 1.2em;"><strong>Centralisation institutionnelle</strong></span>
  <li> <span style="font-size: 1.2em;"><strong>Suivi et évaluation</strong></span> : outil
  <li> <span style="font-size: 1.2em;"><strong>Polyvalence fonctionnelle</strong></span>
</ul>
```

{{2-3}}

```
<div style="color:white; display:flex; align-items:center;">
  <div style="font-size: 1.2em;"><strong>Moodle et consort (LMS) : les moins bons côtés</strong>
</div>
```

```
<ul>
  <li> <span style="font-size: 1.2em;"><strong>Encapsulage institutionnel</strong></span>
```

¹Alt GR + 3 mais vous saviez.

```

    <li> <span style="font-size: 1.2em;"><strong>Interface graphique utilisateur lourde</strong></span>
    <li> <span style="font-size: 1.2em;"><strong>La tendance au tracking overkill</strong></span></li>
</ul>

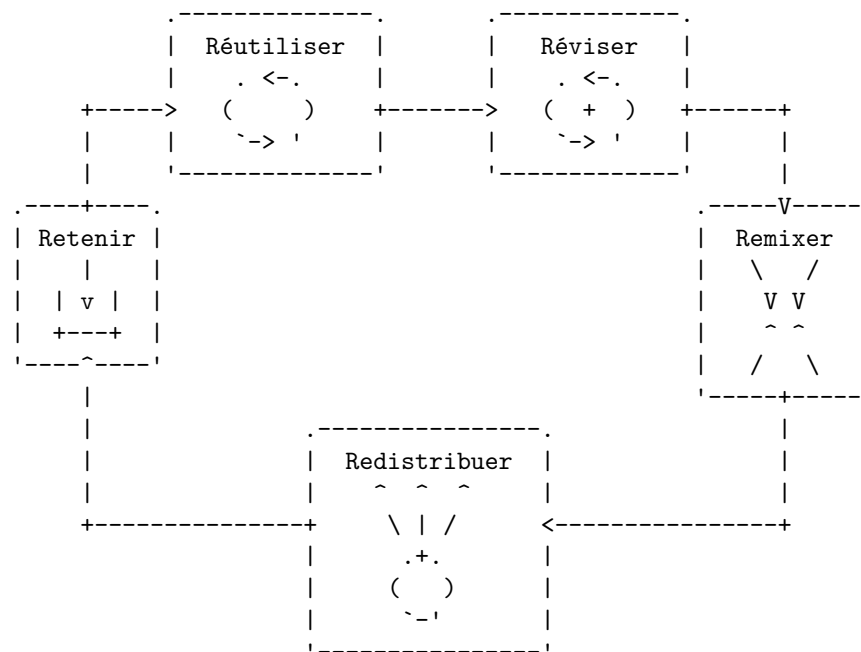
```

Un outil pensé pour l’accessibilité et le partage

Les 5R

--{{0}}--

J’utilise à titre personnel une définition assez simple de ce que sont les REL, celle de David Wiley (cf. La Fabrique REL ainsi que le blog de David Wiley), qui me rappelle les 5W des journalistes. Les 5R font office de critères pour définir si telle ou telle ressource est bien ” libre “. Je ne détaille pas plus, et je vais vous montrer concrètement comment une ressource éducative qui a été faite sur LiaScript permet nativement ces 5 actions.



Retenir (Retain) L’action retain/retenir, c’est :

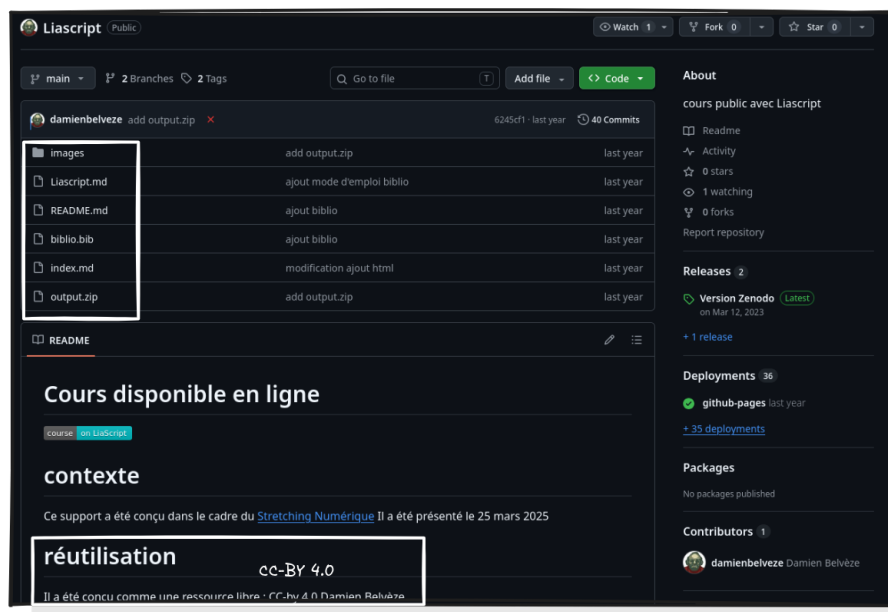
” make, own, and control a copy of the resource ”

– David Wiley (*Defining the “Open” in Open Content and Open Educational Ressources*, 2023)

--{{1}}--

Mettons que je traîne sur GitHub et que je tombe sur **ce cours** (coucou Damien), il est pile dans mon sujet, et son auteur a pris la peine de mentionner de manière explicite la licence de réutilisation, on est sur un CC-BY classique (donc réutilisation permise avec mention de l'auteur).

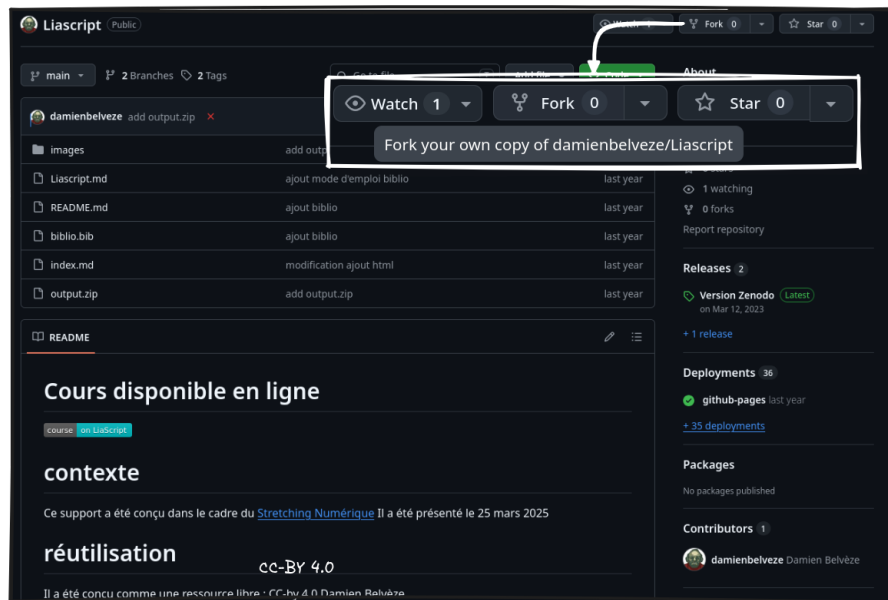
{{1-2}}



--{{2}}--

Le fonctionnement de GitHub est pensé pour ce genre de cas, je peux *forker* le répertoire, c'est-à-dire que j'en crée une copie qui va directement dans la liste de mes répertoires sur mon compte.

{{2-3}}



--{{3}}--

Je lui donne un nom, pourquoi pas une description et je crée le *Fork*.

{{3-4}}

Create a new fork

A *fork* is a copy of a repository. Forking a repository allows you to freely experiment with changes without affecting the original project.

Required fields are marked with an asterisk (*).

Owner * Repository name *

 sjamet8000 /

 Liascript is available.

By default, forks are named the same as their upstream repository. You can customize the name to distinguish it further.

Description

27 / 350 characters

☒ Copy the `main` branch only

Contribute back to damienbelveze/Liascript by adding your own branch. [Learn more.](#)

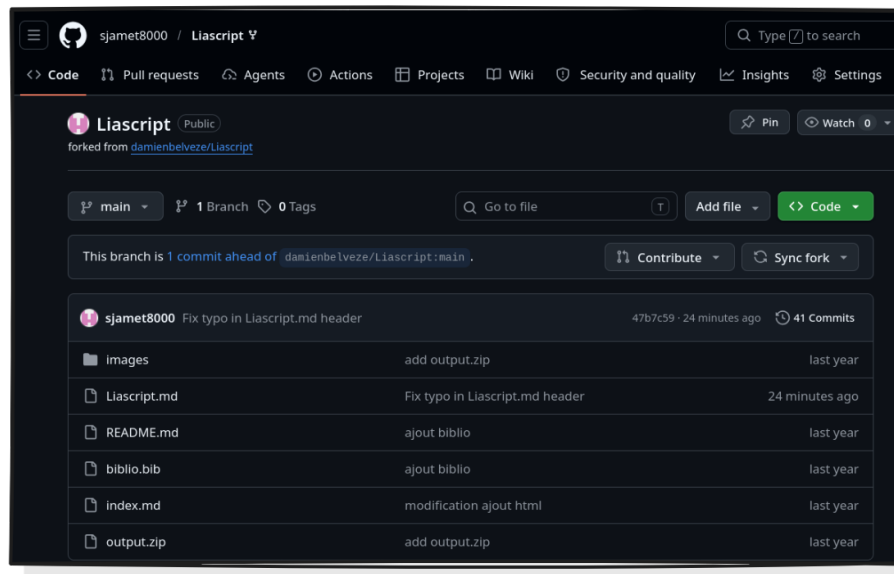
 You are creating a fork in your personal account.

[Create fork](#)

--{{4}}--

Je retrouve le répertoire forké avec tout son contenu, je peux le modifier dans porter atteinte à l'intégrité du répertoire d'origine, tout en conservant son historique, donc une traçabilité des modifications.

{{4-5}}



Réviser (Revise) L'action revise/réviser, c'est :

” edit, adapt, and modify your copy of the resource ”

– David Wiley (*Defining the “Open” in Open Content and Open Educational Ressources*, 2023)

--{{0}}--

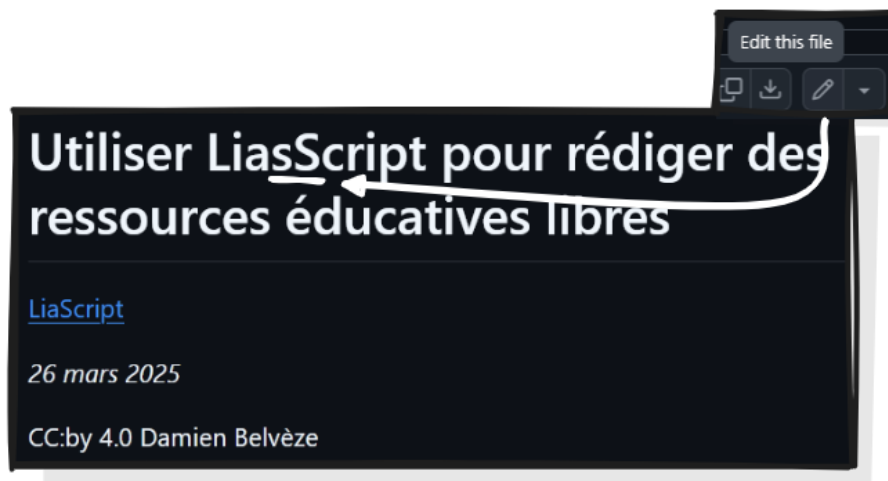
A partir de ce moment-là, je suis libre de faire ce que je veux sur le contenu du cours, ça n'a pas de conséquence sur le répertoire d'origine.

Je peux par exemple corriger une minuscule faute que j'ai repéré dans le titre.

--{{1}}--

Dans GitHub je ne peux pas corriger directement dans l'affichage **Preview**, mais il me suffit de cliquer sur **Edit this file** pour afficher le *Code* qui, lui, est éditabile.

{{1-2}}



--{{2}}--

Je supprime le " s " en trop (je vous invite à aller voir son cours qui est évidemment d'excellente facture).

{{2-3}}

Le *Code* est austère, mais c'est un type d'affichage auquel il faut s'habituer !

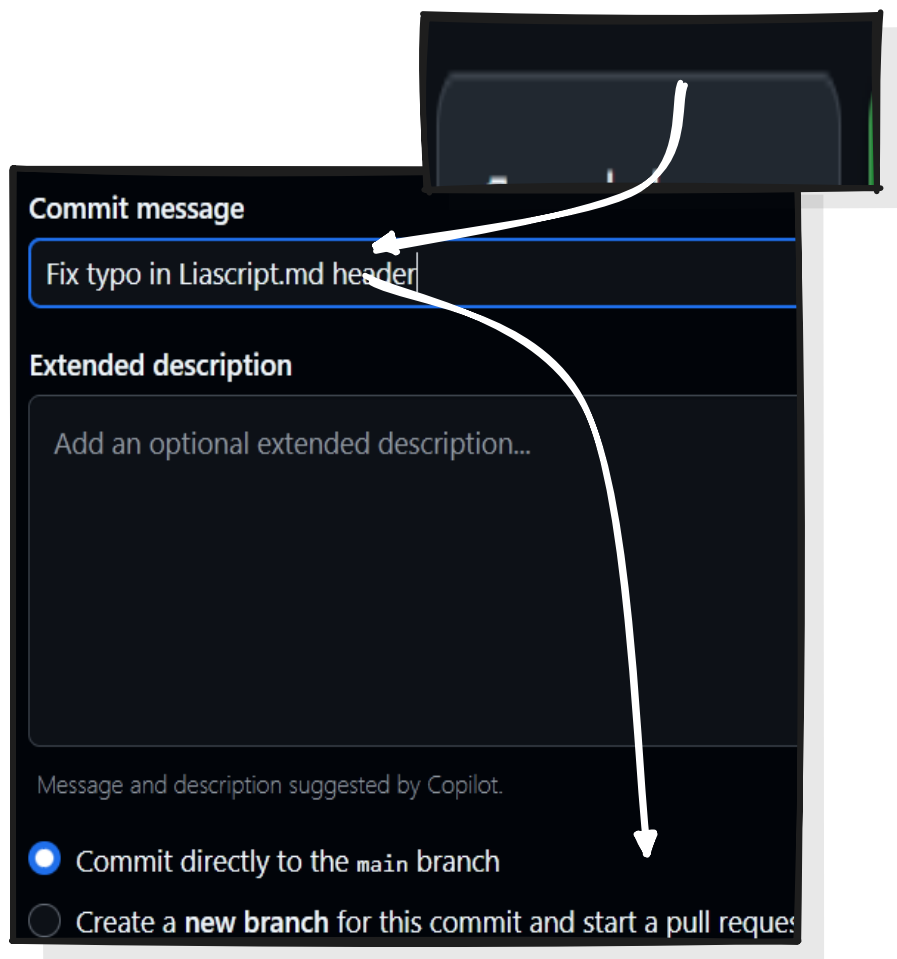
```

52
53  -->
54
55  # Utiliser LiaScript pour rédiger des ressources éducatives libres
56
57  ![[LiaScript](https://raw.githubusercontent.com/LiaScript/LiaScript/
58  https://raw.githubusercontent.com/damienbelveze/Liascript/main/Liascr
59
60  *26 mars 2025*
61
62  CC:by 4.0 Damien Belvèze
63
64  ![[images/logo_SN.png]<!--width="30%"-->

```

{{3-4}}

Autre point de vocabulaire, ne cherchez pas le bouton *save* ici. Pour enregistrer vos modifications, même minimes, cliquez sur **Commit changes...**, décrivez la modification, puis sur **Commite Changes**, voilà vous avez " sauvegardé " !



--{{3}}--

Il n'y a pas de traduction parfaite pour le terme "commit" qu'on peut entendre comme le fait de *valider* une modification.

Remixer (Remixe) L'action reuse/réutiliser, c'est :

"combine your original or revised copy of the resource with other existing material to create something new"

– David Wiley (*Defining the "Open" in Open Content and Open Educational Resources*, 2023)

Ce que tu fais : 1. Tu ouvres le fichier .md 2. Tu colles ta section Git (préparée en amont, markdown prêt) 3. Tu commit 4. Tu rafraîchis la vue LiaScript pour montrer le résultat

Ce que tu dis : > “Aller plus loin : j’ai écrit de mon côté toute une partie sur l’utilisation de Git, qui complète bien ce cours. Je vais l’injecter ici. Je colle, je commit, et... voilà. Le cours a gagné un chapitre entier. **Ça, c’est le Remix :** je fusionne du contenu d’origines différentes pour créer une nouvelle ressource. > > Et notez bien : c’est du markdown. C’est lisible, c’est éditable dans n’importe quel éditeur de texte. Pas de format propriétaire, pas de dépendance à une plateforme.”

Backstage : c’est le bloc le plus long, celui qui peut déborder. Si tu es en retard, tu peux écourter en ne recommitant pas et en montrant juste le markdown source.

Plan B : capture avant/après du rendu LiaScript.

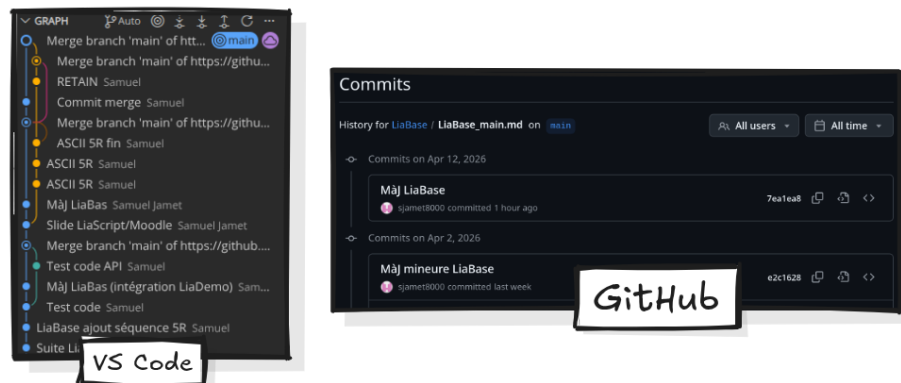
Le suivi des versions

--{{0}}--

Sans doute un des aspects les plus importants quand on travaille sur du long terme ou à plusieurs, c’est de pouvoir s’y retrouver entre plusieurs états d’un même projet, d’éventuellement revenir à un état antérieur, voire de dupliquer un projet entier (à soi ou non) pour le modifier en tant que projet indépendant (faire un **fork**).

{{1-2}}

Ci-dessous l’interface de suivi des versions d’un même projet dans VS Code et



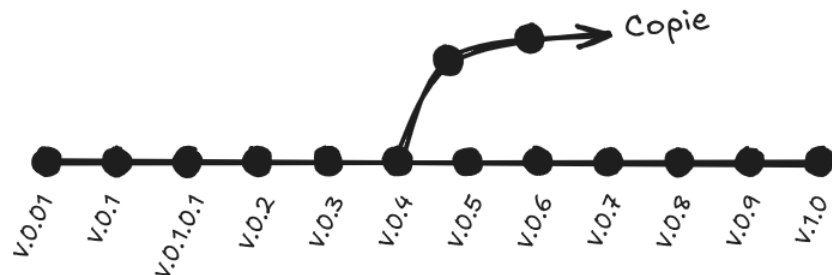
dans GitHub

--{{1}}--

Le suivi des versions implique aussi que votre travail, ou le travail de quelqu’un d’autre qui vous ferait de l’œil est implicitement disponible à la modification et au remixage. sans risque de corruption (on peut toujours revenir à une version antérieure).

{{2}}

Le versioning, une invitation au partage et à la collectivisation rigoureuse des



moyens de productions pédagogiques.

Réutiliser (Reuse) L'action reuse/réutiliser, c'est :

” use your original, revised, or remixed copy of the resource publicly ”

– David Wiley (*Defining the “Open” in Open Content and Open Educational Ressources*, 2023)

A ce stade, je peux d'ores et déjà réutiliser le cours à mon propre compte, si j'estime ne pas avoir besoin d'apporter plus de modifications ou de faire des adaptations supplémentaires.

Redistribuer (Redistribute) L'action redistribute/redistribuer, c'est :

” share copies of your original, revised, or remixed copy of the resource with others ”

– David Wiley (*Defining the “Open” in Open Content and Open Educational Ressources*, 2023)

Ce que tu fais : 1. Sur GitHub, tu ouvres le .md modifié 2. Tu cliques sur “Raw” 3. Tu copies l'URL raw 4. Tu la colles dans le générateur de lien LiaScript (ou directement dans l'URL LiaScript) 5. Tu montres le cours live, utilisable 6. Tu évoques : “ce lien, je peux le poster n'importe où — mon ENT, un mail, Zenodo pour archivage pérenne avec DOI...”

Ce que tu dis : > “Dernière étape : je veux partager ma version remixée. Je récupère le lien raw, je le bascule en URL LiaScript, et j'obtiens un cours vivant, accessible à n'importe qui avec juste un navigateur. Ce lien, je peux le poster où je veux : mon ENT, un mail à mes collègues, ou même Zenodo pour obtenir un DOI et un archivage pérenne. > > **Ça, c'est le Redistribute.** Et remarquez quelque chose d'important : GitHub, qu'on présente souvent comme un outil de développeur, fonctionne ici comme un **espace de partage et de valorisation** de contenus pédagogiques. Exactement comme HAL ou Zenodo, mais avec en plus la dimension éditale.”

Plan B : lien LiaScript déjà généré en amont, prêt à coller si la chaîne casse.

18:00 — 20:00 | Bouclage + transition Ce que tu dis : > “Et maintenant, quelqu’un d’autre peut forker mon cours remixé, le réviser à son tour, le re-remixer, le re-redistribuer. **Le cycle des 5R reprend.** C’est ça, une culture REL vivante : pas une ressource figée qu’on télécharge, mais une ressource qui circule et qui s’enrichit. > > [Transition vers Souplesse] On vient de voir que vos contenus peuvent circuler librement. Mais encore faut-il que l’outil soit capable de porter ce que vous voulez vraiment enseigner. Passons au deuxième argument : la souplesse.”

Questions chat : tu prends 1-2 questions rapides si le timing le permet, sinon tu renvoies à la fin.

Un outil SOUPLE

{{0-1}}

On peut juste écrire du texte, tout simplement...

{{1-2}}



On peut insérer des images...

{{2-3}}

```
+-----+
+-->| Des schémas |
|   +-----+
|
```

```

+-----+      |      +-----+
| On peut faire +---+-->| en ASCII-art |
+-----+      +-----+

```

{{3-4}}

```

                        +-----+
                        +-->| Des schémas |
                        |      +-----+
                        |
+-----+      |      +-----+
| On peut faire +---+-->| en ASCII-art |
+-----+      |      +-----+
                        |
                        |      +-----+
                        +-->| qui deviennent des images en svg |
                        +-----+

```

{{4-5}}

On peut intégrer à peu près tout et n'importe quoi, présentation Genially, tableau Miro, mais surtout... du H5P :-) (si vous voulez rester dans de la REL pure et dure)

{{4-5}}

{{5}}

Il est même possible d'exécuter du code

{{5-6}}

```

const motCle = "bibliothécaires formateurs";
const url = `https://api.archives-ouvertes.fr/search/?q=${encodeURIComponent(motCle)}&rows=3`;

fetch(url)
  .then(response => response.json())
  .then(data => {
    console.log(" Nombre total de résultats dans HAL : " + data.response.numFound.toLocaleString());
    console.log("---");
    console.log(" 3 publications les plus récentes :");
    console.log("");

    data.response.docs.forEach((doc, i) => {
      // On nettoie les chevrons encodés pour l'affichage
      const citation = doc.label_s.replace(/&#x27E8;/g, ' ').replace(/&#x27E9;/g, ' ');
      console.log(`[${i+1}] ${citation}`);
      console.log(`      ${doc.uri_s}`);
      console.log("");
    });
  });

```

```

    });
  })
  .catch(error => {
    console.log(" Erreur : " + error);
  });

"Interrogation de HAL en cours...";

```

Un outil REMPLAÇABLE

Et si LiaScript venait à mourir ?

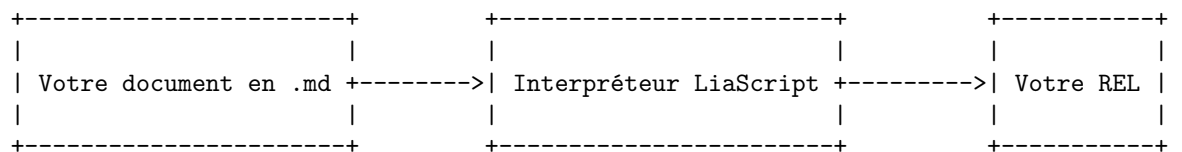
--{{0}}--

LiaScript à la qualité d'être un outil remplaçable, il ne vous prend pas en otage. S'il meurt, on ne perd que la forme, mais pas le fond, pas les éléments, pas les intentions pédagogiques fondamentales. L'outil en tant que tel n'est qu'un interpréteur de votre code, et ce code se trouve être en majeure partie du MarkDown, donc globalement du texte.

{{0-1}}



{{1-2}}



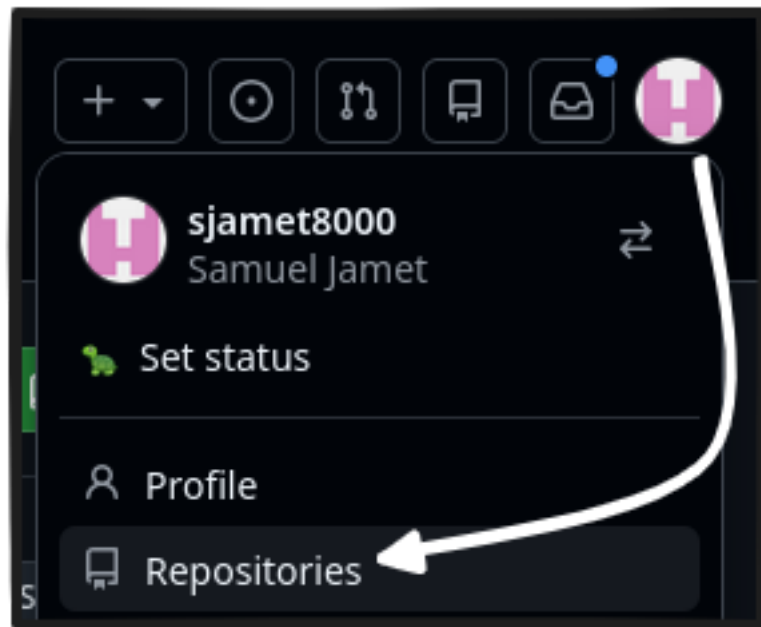
Mise en route

Se créer un compte GitHub

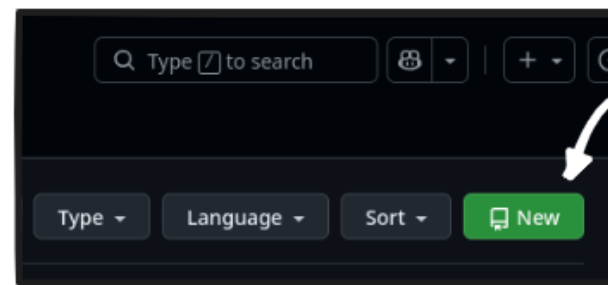
Pour utiliser tranquillement Liascript, la création d'un compte sur GitHub est nécessaire.

Créer un répertoire

{{1-3}} Cliquez sur l'icône de votre compte en haut à droit de l'écran, puis sur



Repositories



{{2-3}} Cliquez sur *New* pour créer un nouveau répertoire.

{{3}} Entrez les informations nécessaires. Evitez les espaces dans le nom du répertoire. la description n'est pas obligatoire, mais elle peut aider à ce que l'on retrouve votre répertoire. Super important : votre répertoire



Sign in to GitHub

Username or email address

Password [Forgot password?](#)

[Sign in](#)

or

 [Continue with Google](#)

 [Continue with Apple](#)

New to GitHub? [Create an account](#)

[Sign in with a passkey](#)

Figure 3: Page de connexion de GitHub

doit impérativement être **public** pour que Liascript puisse l'interpréter.

Create a new repository

Repositories contain a project's files and version history. Have a project elsewhere? [Import a repository](#).
Required fields are marked with an asterisk (*).

1 General

Owner * sjaemet8000 / Repository name * votrenomderepertoire
✔ votrenomderepertoire is available.

Great repository names are short and memorable. How about [sturdy-octo-tribble](#)?

Description

76 / 350 characters

2 Configuration

Choose visibility *
Choose who can see and commit to this repository Public

Add README
READMEs can be used as longer descriptions. [About READMEs](#) On

Add .gitignore
.gitignore tells git which files not to track. [About ignoring files](#) No .gitignore

Add license
Licenses explain how others can use your code. [About licenses](#) Creative Commons Zero v1.0 Universal

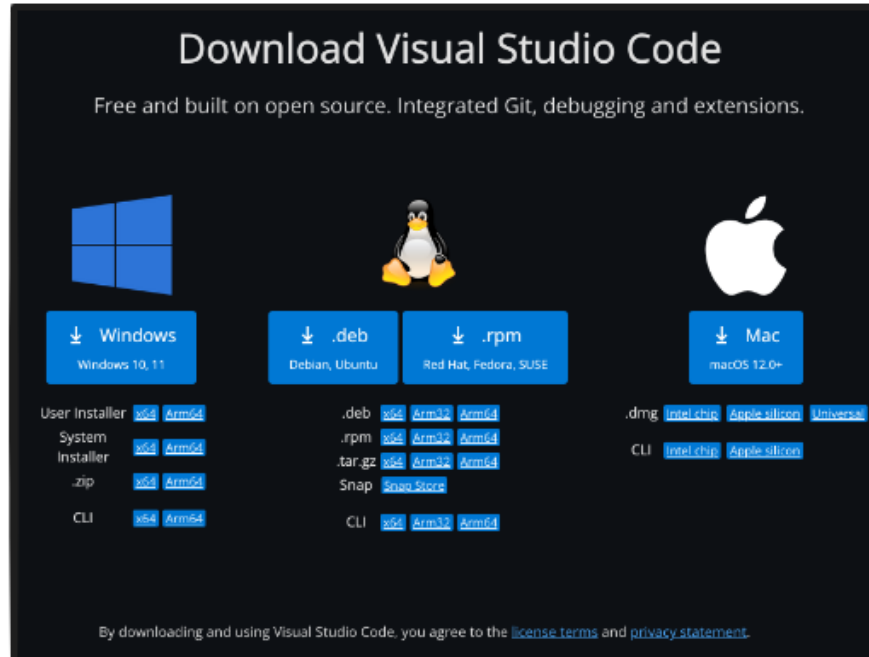
[Create repository](#)

{{4}} Une fois votre répertoire créé, vous pouvez y accéder depuis *Repositories* en cliquant sur l'icone de votre profil. Notez qu'au moment de la création du répertoire, il ne contient rien que la **LICENCE** et le README.md.



Visual Studio Code

Visual Studio Code est un éditeur de texte gratuit et open source maintenu par **Microsoft** (eh oui, sniff). Très utilisé par les développeur.ses, on va plutôt l'utiliser comme simple éditeur de MarkDown, un peu amélioré par le créateur de

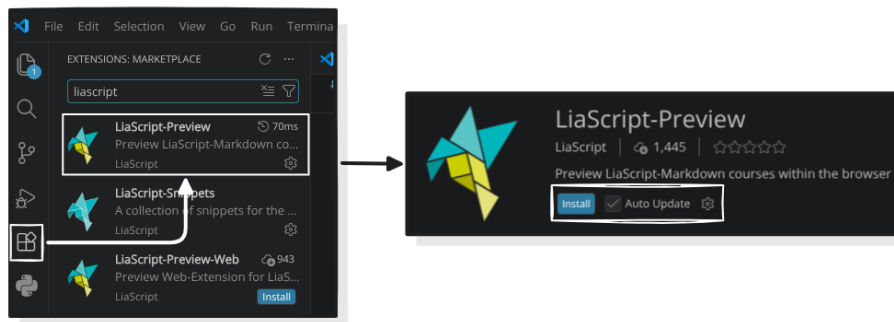


Liascript, grâce à quelques plug in.

Rendez-vous sur la page de téléchargement et procédez à l'installation du logiciel, on se retrouve de l'autre côté.

Installer et activer les plugins Liascript Pour utiliser confortablement Liascript, on va devoir installer deux extensions développées par André Dietrich. Rendez-vous dans le magasin d'extensions (cf. capture ci-dessous ou **Ctrl + Shift + X**).

Entrez **liascript** dans la barre de recherche, normalement trois extensions doivent remonter. Cliquez sur **LiaScript-Preview** puis sur **Install**.



Ce plug in va vous permettre de lancer une prévisualisation de ce que vous êtes en train de produire depuis Visual Code, sur votre navigateur favori, en local, sans connexion internet. Plutôt utile pour vérifier et tester votre cours au fil de l'eau !

Installer et activer le plugin Liascript Snippets Le second plug in se nomme **LiaScript-Snippets**, il n'est pas absolument nécessaire à l'utilisation de LiaScript avec VS Code, mais c'est une aide non négligable quand on débute.

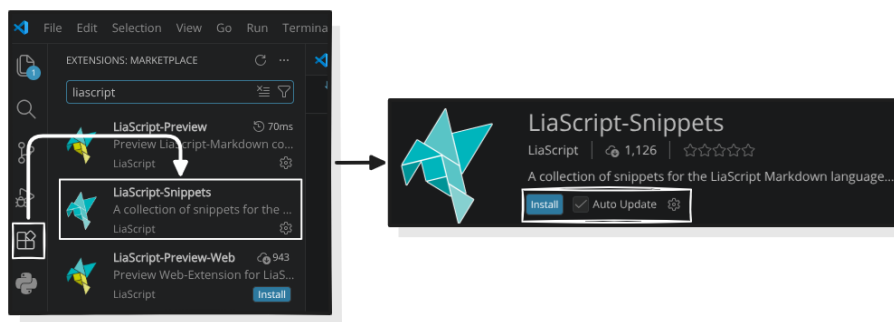


Figure 4: Liascript Snippets

Une fois l'installation faite, ce n'est pas terminé. Sur la page de **LiaScript-Snippets**, on vous demande de copier-coller un morceau de code. Sélectionnez

le morceau de code mis en exergue, puis **Ctrl + C**.

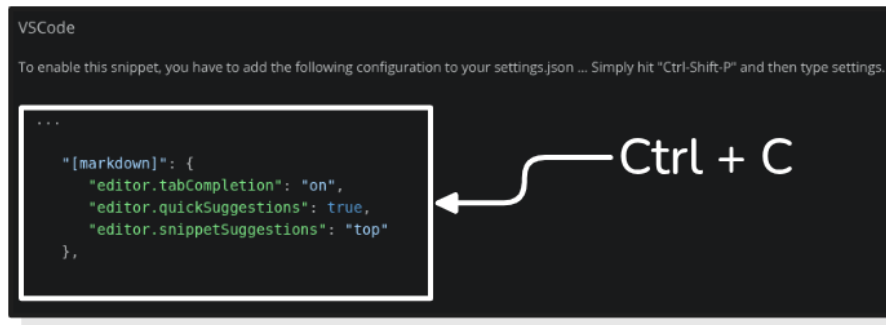


Figure 5: Liascript Snippets suite

Faites **Ctrl + Shift + P** puis entrez `settings` et cliquez sur `Open User Settings (JSON)`, et collez le morceau de code entre les deux accolades (je le remets ci-dessous), puis quittez.

```
"markdown": {  
  "editor.tabCompletion": "on",  
  "editor.quickSuggestions": true,  
  "editor.snippetSuggestions": "top"  
},
```

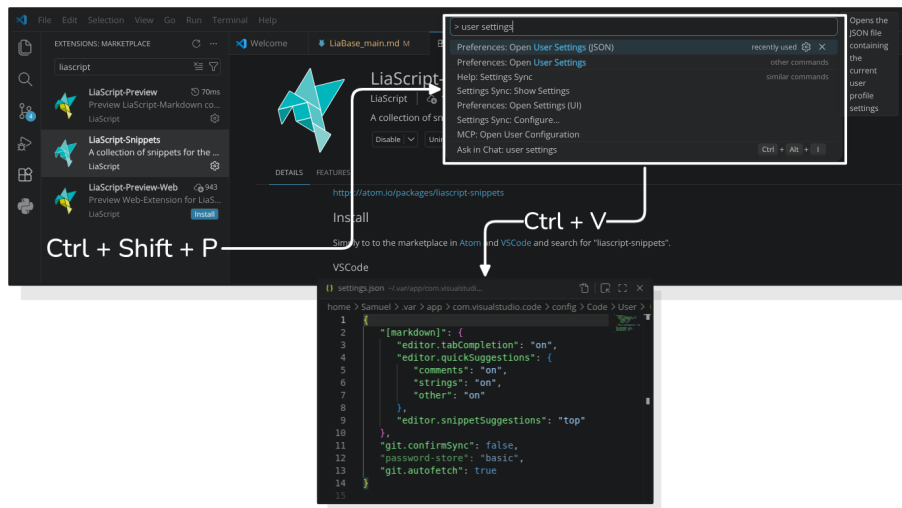


Figure 6: Liascript Snippets suite

LiaScript-Snippets va vous permettre d’“appeler” des fonctionnalités de LiaScript comme des modèles mise en formes spécifiques (tableaux, graph,

etc.), ou des modèles de quizzes, et une miriade de choses super chouettes avec le préfixe `lia`.

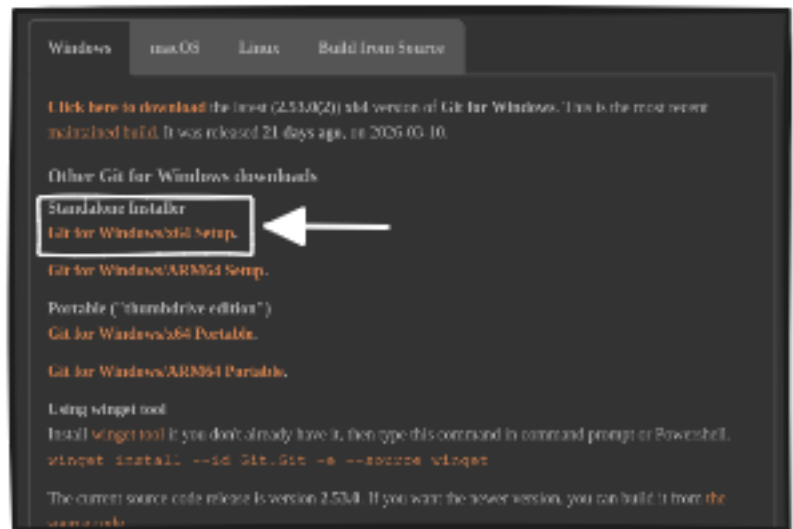
Git



Git est un logiciel de gestion de versions décentralisé développé par Linus Torvalds, le créateur du noyau Linux. Pour en savoir plus sur Git et son fonctionnement, vous pouvez consulter la page Wikipédia qui est bien détaillée.

Installer Git Aller sur le site officiel et de télécharger Git for Windows.

Sélectionnez l'installateur **Git for Windows/x64 Setup** qui devrait fonctionner



dans la plupart des cas.

L'installateur de Git sous Windows a beaucoup d'étapes avec plein de cases à

cocher, laissez toutes les options par défaut et de cliquer sur “Next” jusqu’au bout. Ça ira très bien pour 99 % des usages.

Configurer Git Une fois installé, ouvrez le terminal de commande **Git Bash**.

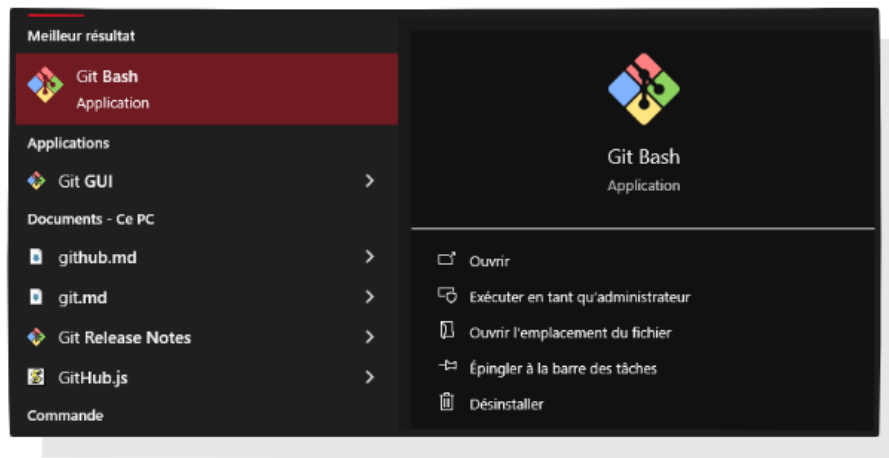


Figure 7: Ouvrir Git Bash

Vérifiez que Git est bien installé en entrant :

```
git --version
```

Si le terminal vous renvoie `git version 2.5x.x.windows` c’est bon.

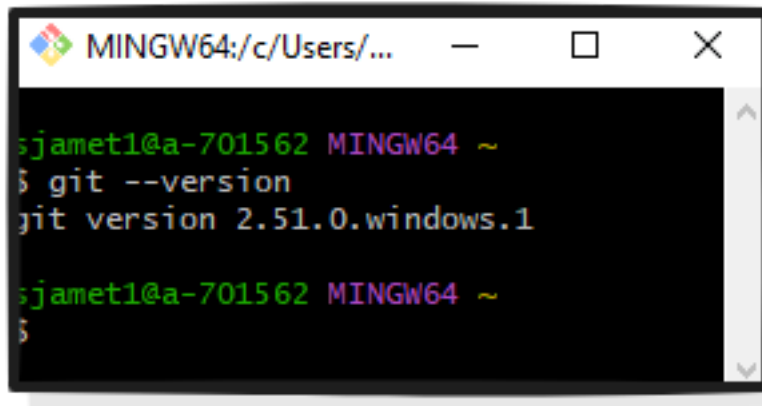


Figure 8: Git version sur Git Bash

C’est là qu’il faut définir l’identité, pour que Git sache qui fait les modifications.

Voici les commandes à entrer (le `--global` est important pour que ça s'applique à tous vos futurs projets) :

Pour le nom (avec les guillemets) :

```
git config --global user.name "Prénom Nom"
```

Pour l'email (idéalement celui que vous utilisez pour GitHub) :

```
git config --global user.email "votre.email@exemple.com"
```

Aujourd'hui, le standard est d'appeler la branche principale `main` (au lieu de l'ancien `master`). Autant configurer ça tout de suite pour éviter des confusions plus tard :

```
git config --global init.defaultBranch main
```

Pour vérifier que tout a bien fonctionné, taper :

```
git config --list
```

Cela affichera votre nom, votre mail et vos réglages. Si vous voyez vos infos, c'est tout bon !

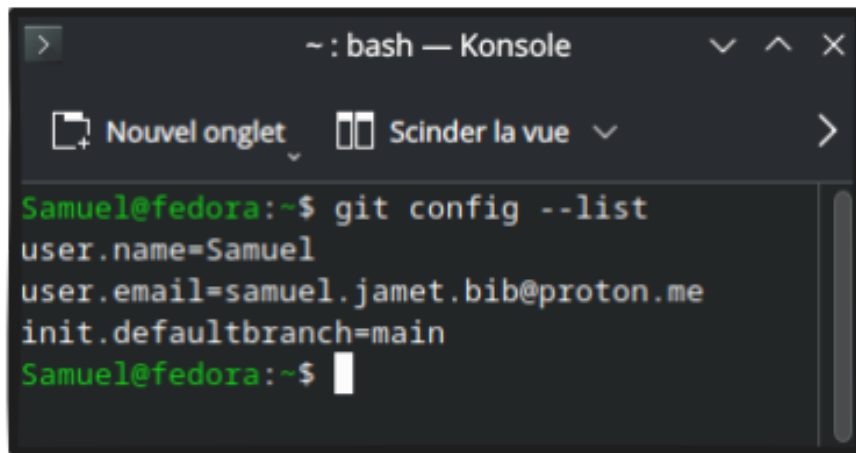
A screenshot of a terminal window titled '~ : bash — Konsole'. The terminal shows the command 'git config --list' being executed. The output is displayed in green text: 'user.name=Samuel', 'user.email=samuel.jamet.bib@proton.me', and 'init.defaultbranch=main'. The prompt 'Samuel@fedora:~\$' is visible at the bottom, followed by a cursor.

Figure 9: Nom, mail et réglages avec git config

Cloner votre répertoire GitHub en local

Rendez-vous dans le dossier dans lequel vous souhaitez synchroniser votre répertoire.

A titre personnel, je clone mes répertoires directement dans le dossier Documents, mais vous pouvez choisir un autre dossier.

Une fois dans le dossier, faites un **clic droit** à l'intérieur puis cliquez sur **Open Git Bash here** pour ouvrir le terminal.

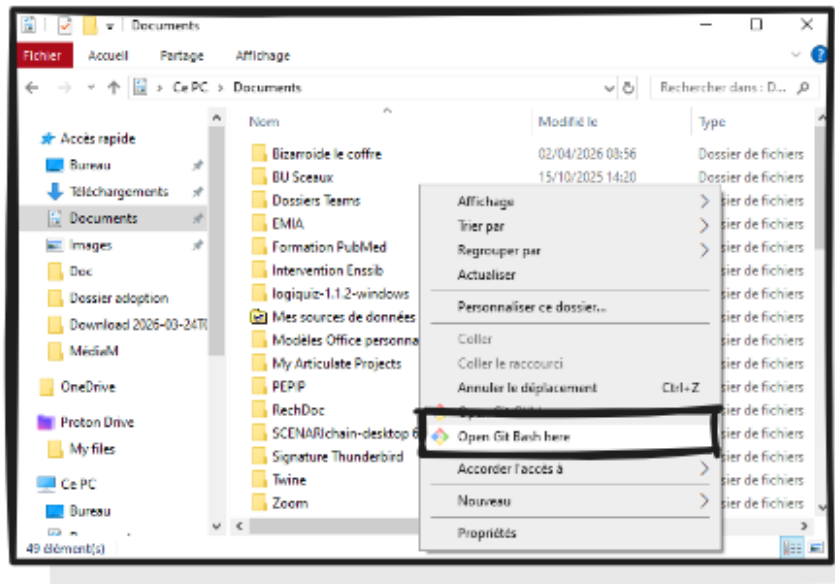


Figure 10: Git bash here

Tapez :

```
git clone https://github.com/nomdevotrecompte/nomdurepertoire.git
```

Vous trouverez l'URL du répertoire en vous rendant... dans votre répertoire bien joué, et en cliquant sur <> code.

Et voilà ! Votre répertoire avec tout son contenu est cloné sur votre PC ! Si c'est votre premier répertoire, seul le README.md et la LICENCE ont été clonées, mais ce n'est qu'un début.

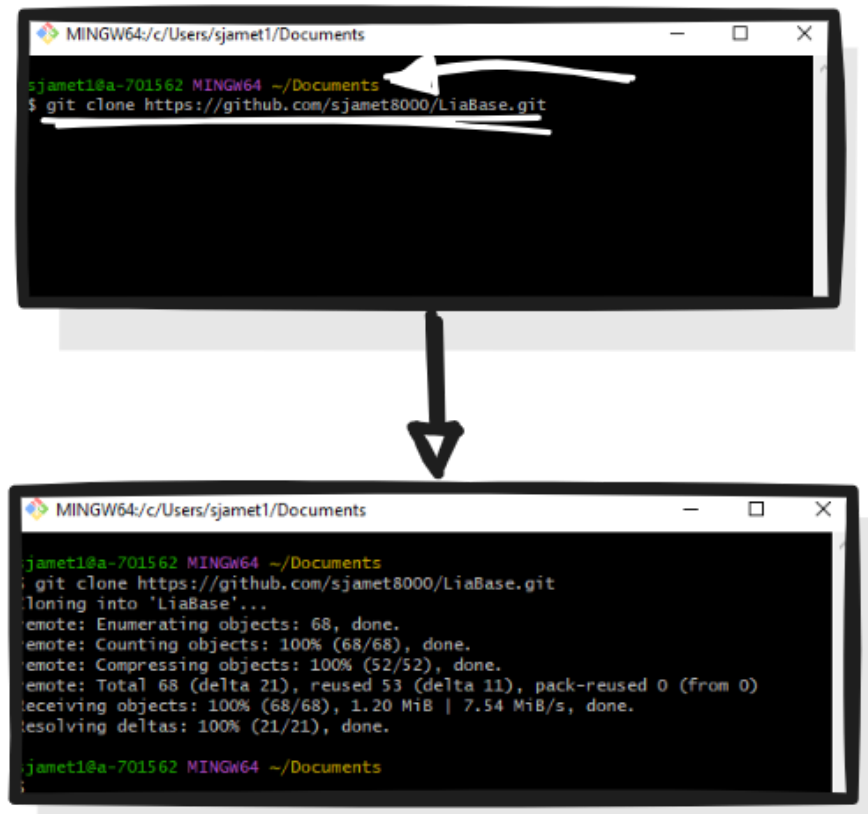


Figure 11: Clonage de répertoire

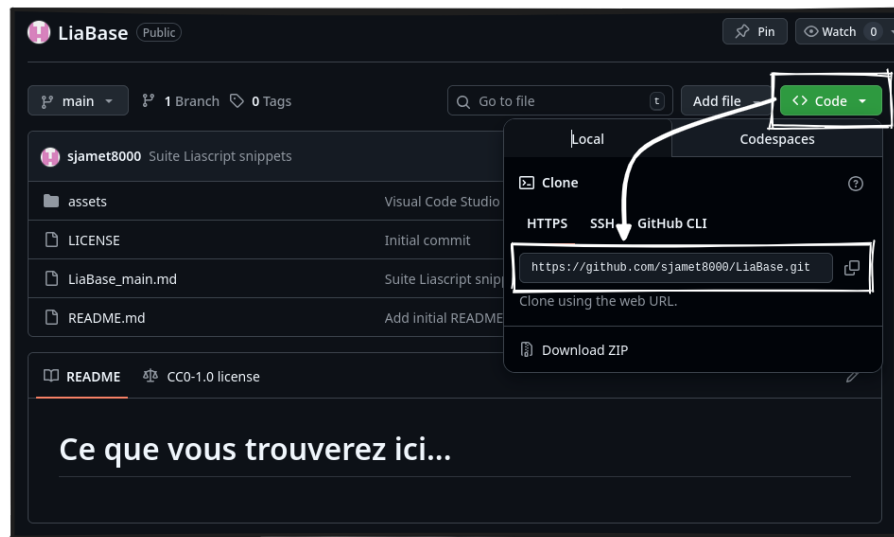


Figure 12: URL GitHub

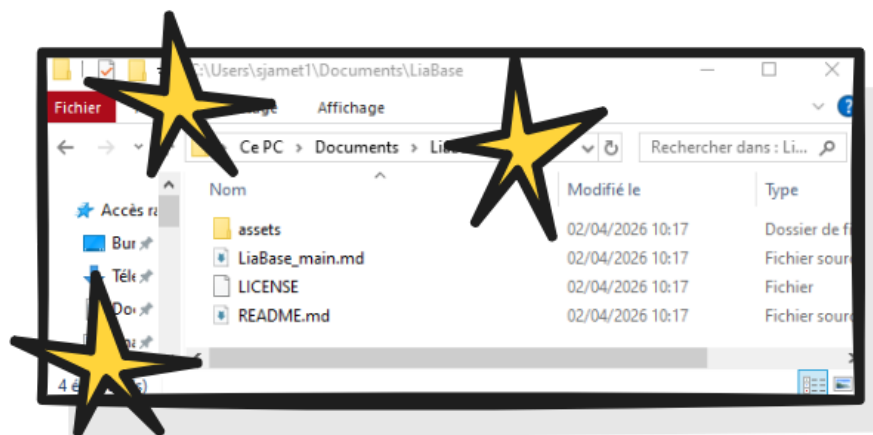
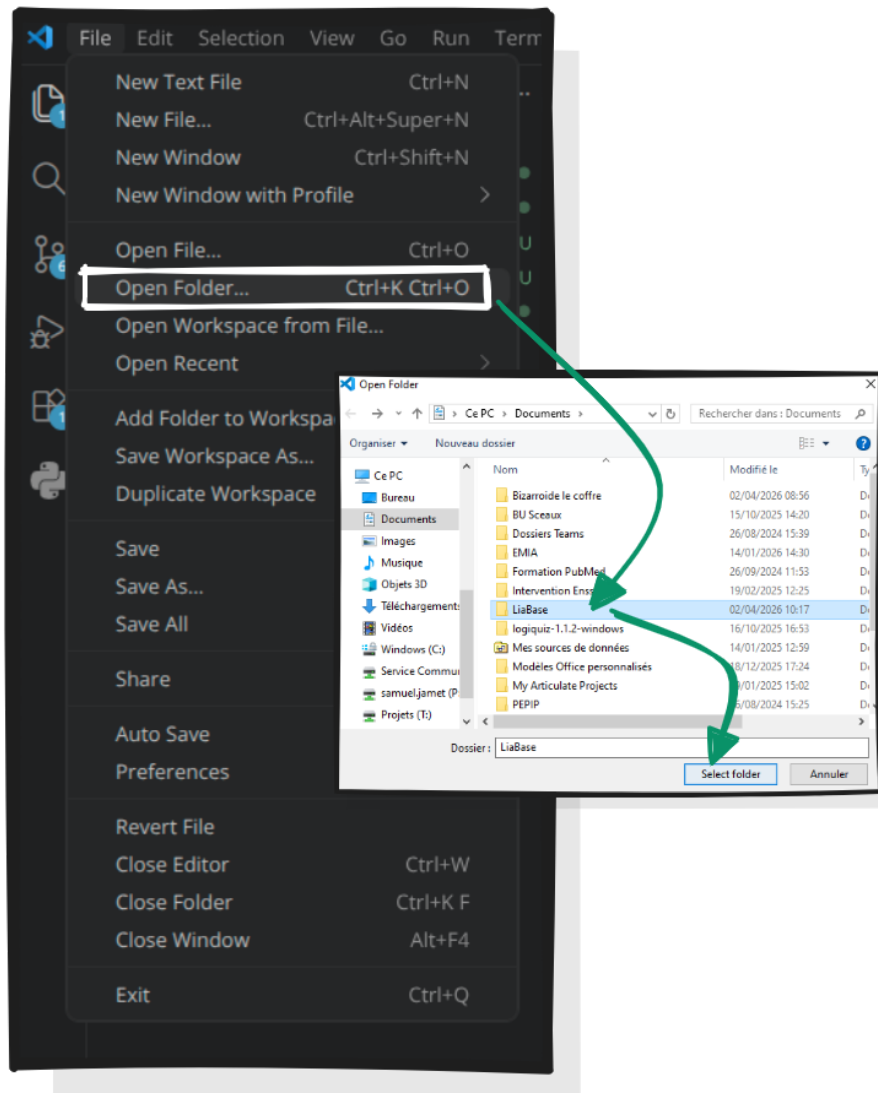


Figure 13: Répertoire en local

Finalisation de la configuration... et premier commit



Rouvrez

VS Code et ouvrez votre répertoire synchronisé : **File > Open Folder...**
> "Nom du répertoire > Select folder" (Sélectionnez simplement le répertoire cloné, ne double-cliquez pas dessus)

Votre arborescence s'affichera sur la partie gauche de la fenêtre de VS Code. Sauf événement cataclismique, elle sera parfaitement identique à celle que vous avez dans l'explorateur de fichiers Windows et votre répertoire dans GitHub.

Sélectionnez le `README.md` pour afficher l'éditeur de texte et voyez ce que ça

donne. C'est visuellement beaucoup plus sobre et plus du tout WYSIWYG, mais on s'y fait.

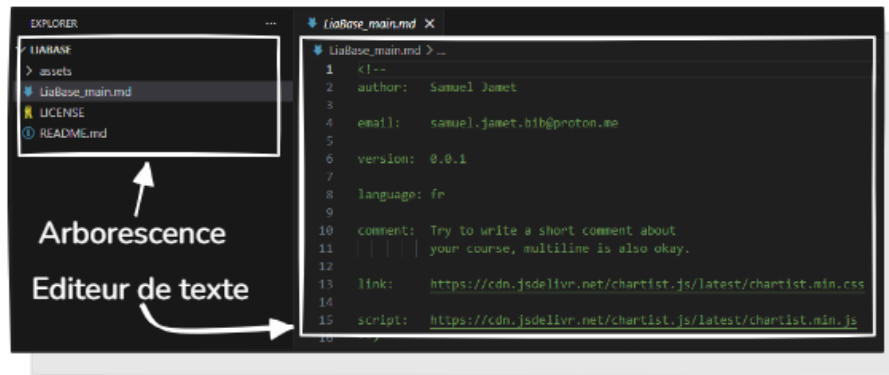
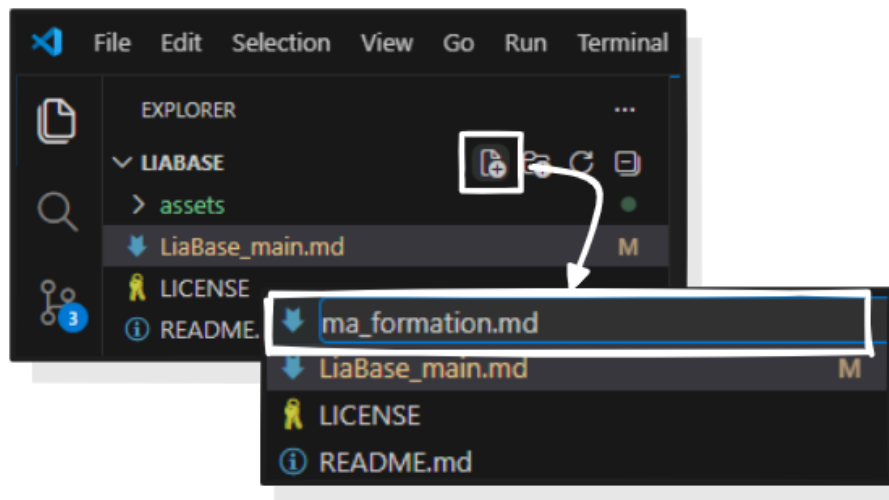


Figure 14: Arborescence répertoire VS Code

Créer votre premier document Evidemment, vous n'allez pas vous mettre à composer dans le README.md (conservez ce fichier pour expliquer comment utiliser votre ressource !)

Dans l'arborescence de votre répertoire, toujours dans VS Code, cliquez sur l'icône New File juste à côté du nom de votre répertoire.

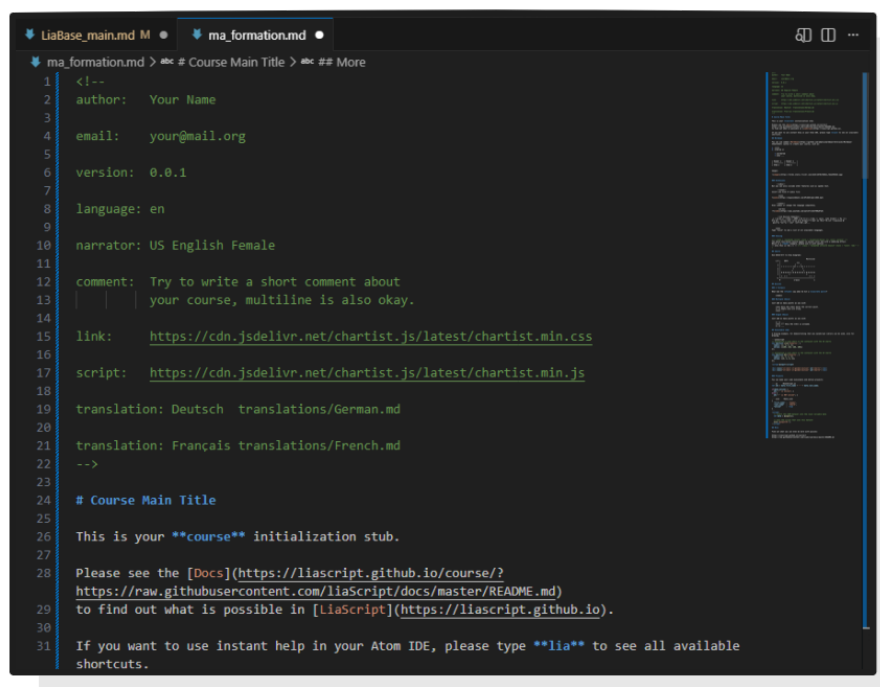


Attention, comme vous le voyez sur la capture, il est indispensable de spécifier le format du fichier nouvellement créé, et ce format, c'est du .md donc le format Markdown. Vous pouvez nommer votre document comme vous le souhaitez, mais ajoutez .md à la fin si vous voulez que ça fonctionne.

Cliquez sur le document pour afficher l'éditeur de texte. Par défaut, il vous propose de générer du code, mais vous ne mangez pas de ce pain-là. Positionnez-vous dans l'éditeur comme vous le feriez pour un éditeur de texte classique et tapez :

`lia-init`

Vous vous souvenez du plug in LiaScript Snippets ? C'est là qu'il entre en action. Appuyez sur **entrée** pour faire apparaître une première base du cours à partir de laquelle vous allez pouvoir avancer, ainsi qu'un certain nombre de métadonnées super utiles.



```
1 <!--
2 author: Your Name
3
4 email: your@mail.org
5
6 version: 0.0.1
7
8 language: en
9
10 narrator: US English Female
11
12 comment: Try to write a short comment about
13 | | | your course, multiline is also okay.
14
15 link: https://cdn.jsdelivr.net/chartist.js/latest/chartist.min.css
16
17 script: https://cdn.jsdelivr.net/chartist.js/latest/chartist.min.js
18
19 translation: Deutsch translations/German.md
20
21 translation: Français translations/French.md
22 -->
23
24 # Course Main Title
25
26 This is your course initialization stub.
27
28 Please see the [Docs](https://liascript.github.io/course/?
29 https://raw.githubusercontent.com/liascript/docs/master/README.md)
30 to find out what is possible in [LiaScript](https://liascript.github.io).
31
32 If you want to use instant help in your Atom IDE, please type lia to see all available
33 shortcuts.
```

Figure 15: Lia-init

Lier VS Code à GitHub Il est temps de vous mettre au travail, vous avez cloné votre répertoire GitHub, vous allez maintenant l'alimenter et le mettre à jour. Pour cela, vous allez lier VS Code à votre compte GitHub (il existe d'autres voies pour synchroniser vos répertoires, ils seront détaillés dans des versions ultérieures de ce guide).

En bas à gauche de la fenêtre de VS Code, cliquez sur **Accounts** puis sur **Backups and Sync Settings**.

Les **Settings Syncs** s'ouvrent en haut de la fenêtre (oui on voyage), cliquez sur

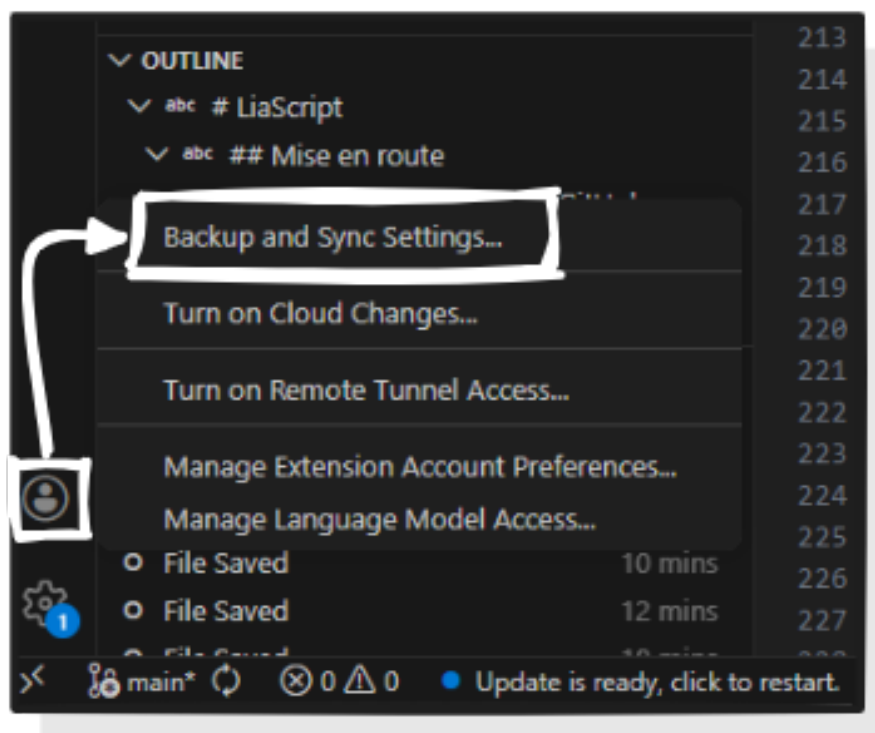


Figure 16: VS Code Accounts

Sign in puis sur Sign in with GitHub

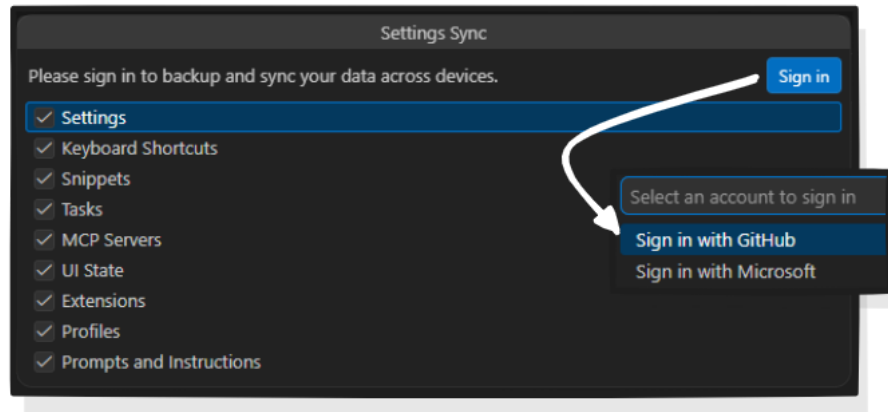
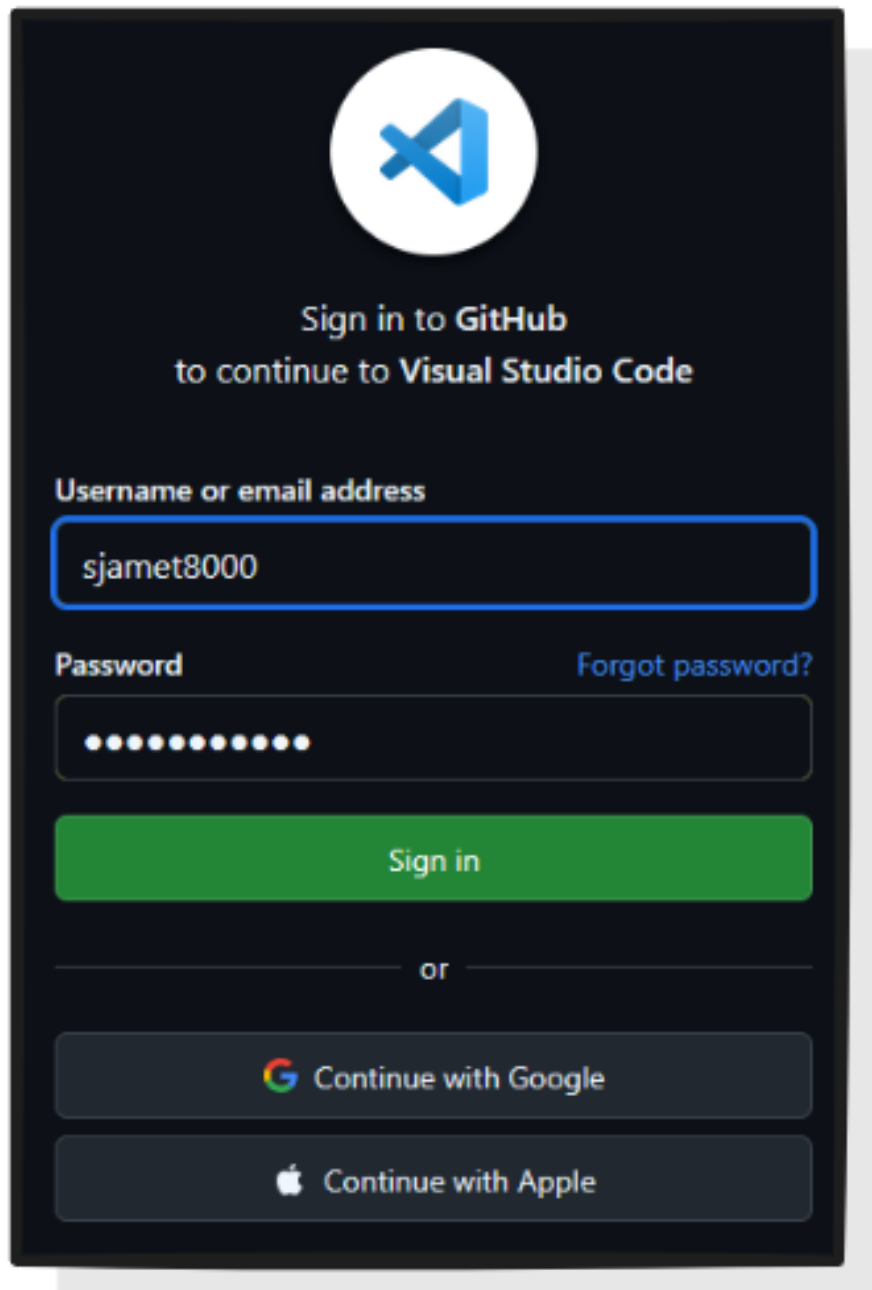


Figure 17: Sign in with GitHub



VS Code va automatiquement ouvrir une page de votre navigateur enregistré par défaut pour vous demander si vous souhaitez vous connecter à GitHub, cliquez sur **Sign in** puis continuez. Si, de retour sur VS Code, le nom de votre

compte s'affiche quand vous cliquez sur **Accounts** c'est bon !

Premier commit Vous avez créé votre premier document et vous souhaitez qu'ils soit synchronisé sur votre répertoire GitHub, vous allez donc faire ce qu'on appelle un *commit*, c'est-à-dire une modification du répertoire qui sera décrite et signée de votre nom.

Cliquez sur **Source Control**(par défaut, parmi les quelques icônes sur le côté gauche de la fenêtre de VS Code)

Premier commit

Figure 18: Premier commit

--{{1}}--

Vous arrivez dans le panneau de contrôle des sources. Juste en dessous du bouton **Commit** un certain nombre de **Changes** sont listés (notamment la création de votre nouveau document.) Cliquez sur le petit + en face de **Changes** les éléments listés deviennent des **Staged Changes** prêts à être envoyé dans votre répertoire GitHub. N'oubliez pas de décrire votre modification/ajout avant de cliquer sur **Commit** puis de cliquer sur **Sync Changes** pour que votre répertoire GitHub soit synchronisé.

Les bases des bases : écrire du texte

--{{1}}--

Si vous n'êtes pas familier avec le Markdown, cette section est là pour y remédier. Pas d'inquiétude, les bases du langage sont très simples et vous aurez vite fait le tour du principal de sa syntaxe. Le plus difficile reste de se défaire de l'accoutumance des éditeurs de texte WYSIWYG (What You See Is What You Get) comme Word.

Ici,

What you see may not be very pleasant...

... mais au moins, ça ne se casse pas.

1. Structurer votre contenu

--{{0}}--

Sur LiaScript, une habitude à prendre, c'est de structurer votre contenu (quel qu'il soit). Entrez #² suivi d'un texte pour créer la page de titre de votre cours, puis structurez de la manière suivante :

²Alt GR + 3 mais vous savez.

Titre principal de votre cours

...

Titre de la première section

...

Titre d'une sous-section

...

Titre de la deuxième section

--{{1}}--

Vous verrez votre table des matières cliquable se structurer sans que vous ayez besoin de faire quoi que ce soit d'autre chaque section, sous-section devient une page du cours.

Il est aussi possible de créer des sous-sections à l'intérieur d'une page de cours

Titre d'une section

Sous-section dans la page

=====

Sous-sous-section dans la même page

2. Mettre en forme votre contenu

--{{0}}--

En Markdown, la mise en forme du texte se fait en entourant ce dernier avec certains caractères que voici :

- **italique** -> *italique*
- ****gras**** -> **gras**
- *****gras et italique ***** -> ***gras et italique***
- *_italique aussi_* -> *italique aussi*
- **__gras aussi__** -> **gras aussi**
- **___gras et italique aussi___** -> ***gras et italique aussi***
- *~barré~* -> *barré*

--{{1}}--

Les mises en forme qui suivent n'existent pas en Markdown de base :

- `~~souligné~~` -> `souligné` (attention avec l'éditeur VS Code votre texte apparaîtra `barré`, mais il sera bien `souligné` dans LiaScript)
- `~~~barré et souligné~~~` -> `~barré et souligné~` (remarque similaire ici, l'affichage du texte dans VS Code peut être confusionnalisant)
- `^en exposant^` -> `en exposant` (comme ça vous allez pouvoir écrire XIX^e et pas XIXe).

More

Find out what you can even do more with quizzes:

<https://liascript.github.io/course/?https://raw.githubusercontent.com/liascript/docs/master/README.md>

Transition Partage → Souplesse > *“On vient de voir que vos contenus peuvent circuler, être repris, modifiés, redistribués. Mais encore faut-il que l’outil soit capable de porter ce que vous voulez enseigner. Parce qu’un outil qui vous enferme dans un format pauvre, c’est une autre forme de dépendance. Regardons jusqu’où LiaScript peut aller.”*

Transition Souplesse → Résilience > *“Vous avez vu qu’on peut aller très loin avec LiaScript. Mais il reste une question : qu’est-ce qui se passe si LiaScript disparaît demain ? Si le projet s’arrête ? C’est là qu’intervient le 3ème argument : vos contenus vous survivent.”*

Clôture > *“Partage, souplesse, résilience. Trois manières différentes de dire la même chose : avec LiaScript, **vos contenus pédagogiques vous appartiennent vraiment**. Et ça, dans un paysage dominé par des plateformes qui captent et enferment, c’est politique.”*